Cluster Rush - Complete Project Tracker

Cluster Rush - Complete Project Tracker

Generated: 2026-09-08
Project: 3D Endless Runner Web Game
Location: ~/vishruth/games/clusterrush/

PROJECT_STATUS.md

Cluster Rush - Project Status

✅ Cleanup Complete

What Was Removed:
1. **Boss Plan** - `boss/` directory and related files
2. **Test Plans** - `test-plan/` directory
3. **Test Results** - `test-results/` directory
4. **Screenshots** - Old test screenshots
5. **Old Workarounds** - Various HTML/JS test files
6. **Debug Scripts** - `final_fix.js`, `diagnose_click.py`, `test_playwright.py`

What Remains:
File	Purpose
`REBUILD_PLAN.md`	**NEW**: Complete rebuild plan (Node.js/Three.js)
`README.md`	Project overview

🎯 NEW PLAN: REBUILD FROM SCRATCH

Technology Stack:
- **Three.js** - 3D rendering (pure JavaScript)
- **Cannon-es** - Physics engine
- **TypeScript** - Type-safe code

- **Node.js** - Local web server

Why This Approach:

Godot WebGL Problem	Three.js Solution
UI invisible in WebGL	DOM-based UI, no bugs
CORS blocks communication	Single-page app
Game frozen	Direct JS control
No debugging	Full DevTools

📋 IMMEDIATE NEXT STEPS

1. Initialize Node.js project with proper structure
2. Create TypeScript configuration
3. Build core game loop (renderer, scene, camera)
4. Implement player movement and physics
5. Add UI system (no CORS issues)
6. Create level loading system
7. Test cross-browser compatibility

Status: ✅ Cleanup complete, ready to rebuild
New Plan: REBUILD_PLAN.md
Target: Pure Node.js/Three.js implementation

==========

PROJECT_STATUS_ENHANCED.md

Cluster Rush - Enhanced Project Status
SPEC-DRIVEN + TEST-DRIVEN DEVELOPMENT APPROACH

✅ ENHANCEMENTS COMPLETE

What Was Added:
1. **Spec-Driven Development Framework** - `SPEC_DRIVEN_DEVELOPMENT.md`

- Complete specification with user stories
- Technical architecture definition
- Success criteria and constraints
- Decision points requiring user input

2. **Test-Driven Development Plan** - `TDD_PLAN.md`
   - Strict RED→GREEN→REFACTOR enforcement
   - Weekly task breakdown with TDD cycles
   - Verification checkpoints and coverage targets
   - Risk mitigation strategies

3. **Enhanced Technical Direction**
   - **Vite** instead of plain Node.js server (faster dev, better DX)
   - **Vitest** testing framework (optimized for Vite/browser code)
   - **TypeScript strict mode** (maximal type safety)
   - **Test pyramid structure** (unit/integration/E2E balance)

What Remains for Implementation:

File	Purpose	Status
`SPEC_DRIVEN_DEVELOPMENT.md`	**Master specification**	✅ Complete
`TDD_PLAN.md`	**TDD implementation guide**	✅ Complete
`REBUILD_PLAN.md`	**Original 14-day plan**	☑️ Retained (reference)
`README.md`	**Project overview**	⏳ Needs update
`PROJECT_STATUS.md`	**Current status**	🔄 This file

🎯 ENHANCED APPROACH: SPEC + TDD

Key Improvements Over Original Plan:

1. **Specification First** (not just planning)
   - Clear user stories with acceptance criteria
   - Technical constraints and success metrics
   - Open decisions flagged for user input

2. **Test-Driven Discipline**
   - No production code without failing test first
   - Weekly coverage targets (80% → 85% → 90%)

- Automation pyramid (unit/integration/E2E)

3. **Modern Tooling**
   - Vite for faster builds and HMR
   - Vitest optimized for browser code
   - TypeScript strict mode from day 1

4. **Risk Management**
   - Identified technical risks with mitigations
   - Verification checkpoints after each phase
   - Performance targets and monitoring

Technology Stack (Enhanced):

Category	Original	Enhanced
Build Tool	Node.js + `serve`	**Vite** (faster dev, better DX)
Testing	Manual testing	**Vitest** + **Puppeteer**
Type Safety	Typescript	**TypeScript strict mode**
Dev Experience	Basic	**Hot module replacement**

Why This Enhancement:

Original Limitation	Enhanced Solution
Vague success criteria	Quantifiable metrics (FPS, coverage, load time)
Ad-hoc testing	TDD enforcement with coverage targets
Reactive development	Proactive spec-driven planning
Manual verification	Automated validation checkpoints

📋 IMMEDIATE NEXT STEPS (TDD-ENFORCED)

Day 1: Infrastructure with Tests First

1. **Initialize project with testing** (`npm init`, Vite config)
   - Red: Test that `npm run test` runs
   - Green: Configure Vitest with TypeScript
   - Verify: Empty test suite passes

2. **TypeScript strict mode**
   - Red: Test compilation fails without types
   - Green: Configure `tsconfig.json` strict options
   - Verify: `npm run build` succeeds

3. **Three.js rendering test**
   - Red: Test WebGL renderer creation fails
   - Green: Minimal Three.js renderer setup
   - Verify: Canvas appears, test passes

Week 1 Focus: Core Systems (80% coverage target)
- Game loop with frame timing
- Input system with keyboard detection
- Basic scene management
- Physics system integration

Week 2 Focus: Gameplay (85% coverage target)
- Player movement and jump mechanics
- Collision detection and scoring
- UI system (DOM-based, no CORS)
- Game state management

Week 3 Focus: Polish & Optimization (90% coverage target)
- Performance optimization
- Cross-browser compatibility
- Final integration testing
- Documentation completion

🧪 TESTING STRATEGY

Test Pyramid:
- **80% Unit Tests**: Individual functions/systems
- **15% Integration Tests**: System interactions
- **5% E2E Tests**: Complete user flows

Coverage Requirements:
- **Phase 1 (Week 1)**: 80%+ branch coverage
- **Phase 2 (Week 2)**: 85%+ branch coverage

- **Phase 3 (Week 3)**: 90%+ branch coverage

Critical Paths (100% coverage required):
1. Game initialization and cleanup
2. Input → Player movement chain
3. Collision detection → Game logic
4. State transitions (menu→play→gameOver)

📊 SUCCESS METRICS (Testable)

Technical Metrics:
- ✅ 90%+ test coverage (branch level)
- ✅ 60 FPS maintained under load
- ✅ <3 second initial load time
- ✅ No memory leaks in extended gameplay

User Experience Metrics:
- ✅ Input latency < 100ms
- ✅ All UI elements functional cross-browser
- ✅ No console errors/warnings
- ✅ Intuitive controls (WASD + SPACE)

Development Metrics:
- ✅ TDD compliance on all production code
- ✅ Daily test suite passes
- ✅ Weekly coverage reports
- ✅ No broken functionality in main branch

🔄 STATUS TRACKING

Current Status:
- **Specification**: ✅ Complete (`SPEC_DRIVEN_DEVELOPMENT.md`)
- **TDD Plan**: ✅ Complete (`TDD_PLAN.md`)
- **Project Setup**: ⏳ Ready to start
- **Implementation**: ⏳ Not started

Blockers:
- None - Ready for implementation

Dependencies:
- User input on open questions in specification
- Development environment ready (Node.js installed)

✅ VERIFICATION CHECKLIST

Before Starting Implementation:
- [ ] User has reviewed and approved specification
- [ ] Open questions in spec have been addressed
- [ ] Development environment confirmed working
- [ ] TDD discipline agreed upon

After Each Task Completion:
- [ ] Tests written first (RED phase verified)
- [ ] Implementation passes tests (GREEN verified)
- [ ] All existing tests still pass (no regression)
- [ ] Coverage requirement met for phase
- [ ] Manual browser test confirms functionality

Status: ✅ Specification complete, ⏳ Ready for TDD implementation
Direction: Spec-driven + test-driven development
Next Action: Begin Day 1 TDD tasks in `TDD_PLAN.md`
Confidence: High (structured approach with verification)

==========

README_ENHANCED.md

Cluster Rush - Enhanced README
🎮 3D Web Game with Spec-Driven + Test-Driven Development

📖 Overview

Cluster Rush is a 3D endless runner game being rebuilt from Godot WebGL to pure WebGL/Three.js to avoid CORS and rendering bugs. This project follows **spec-driven development** and **strict test-driven development** methodologies.

🚀 Quick Start

Prerequisites
- Node.js v20.11.0 or higher
- Modern browser with WebGL support (Chrome 90+, Firefox 88+, Safari 14+)

Installation & Development
```bash
# Clone repository
git clone
cd clusterrush

# Install dependencies
npm install

# Start development server (with hot reload)
npm run dev

# Run tests (TDD workflow)
npm run test

# Build for production
npm run build

# Preview production build
npm run preview
```

Development Server: http://localhost:5173

📋 Project Structure

```
clusterrush/
├── src/                # Source code
├── tests/              # All test files (unit/integration/e2e)
```

```
├── public/              # Static assets and HTML
├── config/              # Build and tool configurations
├── docs/                # Documentation
└── *.md                 # Planning and specification files
```

Key Documentation Files

File	Purpose
`SPEC_DRIVEN_DEVELOPMENT.md`	Complete specification with user stories
`TDD_PLAN.md`	Test-driven development implementation guide
`REBUILD_PLAN.md`	Original 14-day implementation plan
`PROJECT_STATUS_ENHANCED.md`	Current project status and next steps

🧪 Testing Strategy (TDD Enforcement)

Test Commands

```bash
npm run test          # Run all tests
npm run test:unit     # Unit tests only
npm run test:integration # Integration tests
npm run test:e2e      # Browser E2E tests (Puppeteer)
npm run test:watch    # Watch mode for TDD workflow
npm run test:coverage  # Generate coverage report
```

Coverage Requirements

- **Week 1**: 80%+ branch coverage
- **Week 2**: 85%+ branch coverage
- **Week 3**: 90%+ branch coverage

TDD Workflow

1. **RED**: Write failing test describing expected behavior
2. **Verify RED**: Confirm test fails (not error)
3. **GREEN**: Write minimal code to pass test
4. **Verify GREEN**: Confirm test passes
5. **REFACTOR**: Clean code without changing behavior
6. **Verify REFACTOR**: All tests still pass

Rule: No production code without a failing test first.

🎯 Success Criteria

Technical Metrics
- ✅ 90%+ test coverage (branch level)
- ✅ 60 FPS maintained under load
- ✅ <3 second initial load time
- ✅ No memory leaks in extended gameplay

User Experience Metrics
- ✅ Input latency < 100ms
- ✅ All UI elements functional cross-browser
- ✅ No console errors/warnings
- ✅ Intuitive controls (WASD + SPACE)

🔧 Technology Stack

Core Dependencies
- **Three.js v0.162.0** - 3D rendering engine
- **Cannon-es v0.20.0** - Physics simulation
- **TypeScript v5.4.5** - Type-safe development

Development Tools
- **Vite v5.3.0** - Fast builds and dev server
- **Vitest v1.6.0** - Testing framework
- **Puppeteer v22.6.0** - Browser automation

📖 Development Phases

Phase 1: Core Infrastructure (Week 1)
- Project setup with TDD from day 1
- Core game loop and input system
- Basic Three.js rendering pipeline
- **Target**: 80% test coverage

Phase 2: Gameplay Systems (Week 2)
- Player movement and jump mechanics
- Physics integration with collision detection
- Scoring system and game state management
- **Target**: 85% test coverage

Phase 3: Polish & Optimization (Week 3)
- UI system with DOM integration (no CORS!)
- Performance optimization and cross-browser testing
- Final integration and documentation
- **Target**: 90% test coverage

🚨 Important Notes

Why Not Godot WebGL?
- **WebGL Bug**: Control UI nodes don't render in Godot 4.x WebGL
- **CORS Barrier**: Can't communicate between HTML overlay and Godot engine
- **Frozen State**: Game engine loads but doesn't accept inputs
- **Solution**: Pure JavaScript/TypeScript with Three.js avoids all these issues

Key Architecture Decisions
1. **DOM-based UI**: HTML overlay instead of Canvas UI (no CORS issues)
2. **Single-page app**: No iframe isolation, direct DOM access
3. **Test-first approach**: Every feature developed with failing test first
4. **TypeScript strict mode**: Maximum type safety from the beginning

🤝 Contributing

Development Workflow
1. Always start with a failing test (RED)
2. Write minimal implementation (GREEN)
3. Clean up code (REFACTOR)
4. Verify all tests pass before commit
5. Check coverage meets phase requirements

Code Standards
- TypeScript strict mode compliance
- JSDoc comments for public APIs
- Consistent naming conventions
- No console errors/warnings in production

📄 License
MIT License - see LICENSE file for details

🙋 Getting Help

- Review `SPEC_DRIVEN_DEVELOPMENT.md` for detailed specifications
- Check `TDD_PLAN.md` for implementation guidance
- Run tests after any changes

Project Status: ✅ Planning complete, ⏳ Ready for TDD implementation
Development Approach: Spec-driven + Test-driven development
Expected Timeline: 3 weeks (14 working days)
Confidence Level: High (structured approach with verification checkpoints)

==========

REBUILD_PLAN.md

Cluster Rush - Complete Node.js/JavaScript Rebuild Plan
From Godot WebGL to Pure Node.js Web Game

🎯 **CURRENT STATE ANALYSIS**

What We Have Now:
- **Game Engine**: Godot 4.x (WebGL export)
- **Current Issue**: UI invisible, CORS blocks communication, game frozen
- **Export**: `Builds/WebGL/` - 39MB WASM + 280KB JS + HTML
- **Web Server**: `webgl_server.py` - Python server for WebGL
- **Attempted Fixes**: HTML overlay, Godot code workarounds, all CORS-limited

Why Godot Failed:
1. **WebGL Bug**: Control UI nodes don't render in Godot 4.x WebGL
2. **CORS Barrier**: Can't communicate between HTML overlay and Godot engine
3. **Frozen State**: Game engine loads but doesn't accept inputs
4. **Multiple Export Attempts**: Same issues every time

🚀 **RECOMMENDED APPROACH: PURE NODE.JS + THREE.JS**

Why NOT Godot:
- ❌ WebGL export bugs are fundamental (not fixable via config)
- ❌ CORS restrictions block cross-frame communication
- ❌ UI rendering failures persist across export versions
- ❌ No reliable way to start the actual game

Why THREE.JS + Node.js:
- ✅ Full JavaScript control - no CORS issues within same page
- ✅ Direct DOM manipulation - no iframe isolation
- ✅ Physics via Cannon-es or Ammo.js
- ✅ Local Node.js server - no web deployment needed
- ✅ WebGL rendering - same performance as Godot
- ✅ Pure code - easy debugging and iteration

📋 **DETAILED IMPLEMENTATION PLAN**

PHASE 1: PROJECT SETUP (Day 1)

Step 1.1: Initialize Node.js Project
```bash
cd ~/vishruth/games/clusterrush
npm init -y
```

Step 1.2: Install Dependencies
```bash
npm install three @types/three cannon-es @types/cannon-es serve
npm install -D typescript ts-node nodemon
```

Step 1.3: Create Project Structure
```
clusterrush/
├── src/
│   ├── main.ts          # Entry point
│   ├── game.ts          # Game class
│   ├── player.ts        # Player logic
│   ├── physics.ts       # Physics engine
```

```
│   ├── input.ts           # Input handling
│   ├── ui.ts              # UI system
│   ├── scene.ts           # Scene management
│   └── config.ts          # Game settings
├── assets/
│   ├── models/            # 3D models (GLTF)
│   ├── textures/          # Textures
│   └── sounds/            # Audio files
├── public/
│   └── index.html         # Main HTML page
├── package.json
├── tsconfig.json
├── server.config.js       # Node.js server config
└── README.md
```

PHASE 2: CORE ENGINE (Day 2-3)

Step 2.1: Main Game Loop (src/main.ts)

```typescript
import { Game } from './game';

const game = new Game();
game.init();
game.start();
```

Step 2.2: Game Class (src/game.ts)

```typescript
import * as THREE from 'three';
import { Input } from './input';
import { Physics } from './physics';
import { UI } from './ui';
import { Player } from './player';

export class Game {
  private scene: THREE.Scene;
  private camera: THREE.PerspectiveCamera;
```

```
  private renderer: THREE.WebGLRenderer;
  private input: Input;
  private physics: Physics;
  private ui: UI;
  private player: Player;
  private clock: THREE.Clock;
  private isRunning: boolean;

  constructor() {
    this.scene = new THREE.Scene();
    this.camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1,
1000);
    this.renderer = new THREE.WebGLRenderer({ antialias: true });
    this.input = new Input();
    this.physics = new Physics();
    this.ui = new UI();
    this.player = new Player();
    this.clock = new THREE.Clock();
    this.isRunning = false;
  }

  init(): void {
    this.setupRenderer();
    this.setupLights();
    this.setupCamera();
    this.setupEventListeners();
  }

  start(): void {
    this.isRunning = true;
    this.animate();
  }

  private setupRenderer(): void {
    this.renderer.setSize(window.innerWidth, window.innerHeight);
    this.renderer.setPixelRatio(window.devicePixelRatio);
    document.body.appendChild(this.renderer.domElement);
  }

  private setupLights(): void {
```

```
    const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
    this.scene.add(ambientLight);

    const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
    directionalLight.position.set(10, 20, 10);
    this.scene.add(directionalLight);
  }

  private animate(): void {
    if (!this.isRunning) return;

    requestAnimationFrame(() => this.animate());

    const deltaTime = this.clock.getDelta();
    this.update(deltaTime);
    this.render();
  }

  private update(deltaTime: number): void {
    this.input.update();
    this.player.update(deltaTime);
    this.physics.update(deltaTime);
    this.ui.update();
  }

  private render(): void {
    this.renderer.render(this.scene, this.camera);
  }
}
```

PHASE 3: GAMEPLAY SYSTEMS (Day 4-6)

Step 3.1: Player System (src/player.ts)

```typescript
import * as THREE from 'three';
import { Input } from './input';
```

```
export class Player {
  private mesh: THREE.Group;
  private velocity: THREE.Vector3;
  private speed: number;
  private jumpForce: number;
  private isGrounded: boolean;

  constructor() {
    this.velocity = new THREE.Vector3(0, 0, 0);
    this.speed = 5;
    this.jumpForce = 10;
    this.isGrounded = true;

    // Create player mesh
    this.mesh = new THREE.Group();

    const geometry = new THREE.BoxGeometry(1, 1, 1);
    const material = new THREE.MeshStandardMaterial({ color: 0x00ff00 });
    const cube = new THREE.Mesh(geometry, material);

    this.mesh.add(cube);
  }

  update(deltaTime: number): void {
    // Movement
    if (this.input.isPressed('w')) {
      this.velocity.z = -this.speed;
    }
    if (this.input.isPressed('s')) {
      this.velocity.z = this.speed;
    }
    if (this.input.isPressed('a')) {
      this.velocity.x = -this.speed;
    }
    if (this.input.isPressed('d')) {
      this.velocity.x = this.speed;
    }

    // Jump
    if (this.input.isPressed(' ') && this.isGrounded) {
```

```
      this.velocity.y = this.jumpForce;
      this.isGrounded = false;
    }

    // Apply physics
    this.velocity.y -= 9.81 * deltaTime; // Gravity
    this.mesh.position.add(this.velocity.clone().multiplyScalar(deltaTime));

    // Ground collision
    if (this.mesh.position.y <= 0) {
      this.mesh.position.y = 0;
      this.velocity.y = 0;
      this.isGrounded = true;
    }
  }

  getMesh(): THREE.Group {
    return this.mesh;
  }
}
```

PHASE 4: LEVEL SYSTEM (Day 7-8)

Step 4.1: Level Manager

```typescript
import * as THREE from 'three';

export class LevelManager {
  private levelData: any[] = [];
  private currentLevel: number = 0;

  loadLevel(levelNumber: number): void {
    this.currentLevel = levelNumber;

    // Load level geometry
    const loader = new THREE.GLTFLoader();
    loader.load(`/assets/levels/level_${levelNumber}.gltf`, (gltf) => {
```

```
      scene.add(gltf.scene);
    });
  }

  getObstacles(): THREE.Mesh[] {
    // Return obstacle meshes for collision
    return [];
  }
}
```

PHASE 5: PHYSICS & COLLISION (Day 9-10)

Step 5.1: Physics Engine

```typescript
import * as CANNON from 'cannon-es';

export class Physics {
  private world: CANNON.World;
  private bodies: CANNON.Body[] = [];

  constructor() {
    this.world = new CANNON.World();
    this.world.gravity.set(0, -9.82, 0);
  }

  update(deltaTime: number): void {
    this.world.step(1 / 60, deltaTime, 3);
  }

  addBody(body: CANNON.Body): void {
    this.world.addBody(body);
  }
}
```

PHASE 6: UI SYSTEM (Day 11)

Step 6.1: HTML-based UI (no CORS issues!)

```typescript
export class UI {
  private scoreEl: HTMLElement;
  private levelEl: HTMLElement;
  private menuEl: HTMLElement;
  private hudEl: HTMLElement;

  constructor() {
    this.setupUIElements();
  }

  private setupUIElements(): void {
    // Create menu
    this.menuEl = document.createElement('div');
    this.menuEl.innerHTML = `
```

CLUSTER RUSH

START GAME

```
    `;
    document.body.appendChild(this.menuEl);

    // Create HUD (hidden initially)
    this.hudEl = document.createElement('div');
    this.hudEl.style.display = 'none';
    this.hudEl.innerHTML = `
```

SCORE: 0

```
    `;
```

```
    document.body.appendChild(this.hudEl);

    // Setup event listeners
    document.getElementById('start-btn')?.addEventListener('click', () => {
      this.menuEl.style.display = 'none';
      this.hudEl.style.display = 'block';
    });
  }
}
```

PHASE 7: BUILD & TEST (Day 12-14)

Build Commands

```bash
npm run build    # Compile TypeScript
npm run serve    # Start local server on port 8080
```

Test Checklist

- [ ] Game loads in browser
- [ ] Menu appears
- [ ] Click START → HUD appears
- [ ] Player moves with WASD
- [ ] Jump with SPACE
- [ ] Score updates
- [ ] Cross-browser compatible

📊 **IMPLEMENTATION TIMELINE**

Day	Phase	Deliverable
1	Setup	Node.js project, dependencies
2-3	Core Engine	Game loop, renderer
4-6	Gameplay	Player, input, UI
7-8	Level System	Level loading

9-10	Physics	Physics engine
11	UI	HTML-based UI
12	Build	TypeScript build
13-14	Testing	Manual testing

Total Time: 14 days (2 weeks)

✅ **SUCCESS CRITERIA**

- [ ] Game loads in browser
- [ ] HTML UI displays correctly
- [ ] Game accepts keyboard inputs
- [ ] Player moves and jumps
- [ ] No CORS errors
- [ ] Cross-browser compatible

Ready to start implementation when you give the go-ahead.

==========